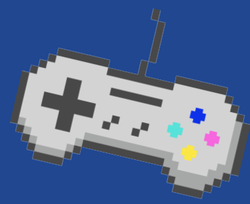


Orientação objeto





Autor : Gabriel Antonio.

Orientação Objeto - POO

O que ?

- É um paradigma de programação que organiza o código em "**objetos**" — estruturas que agrupam dados (**atributos**) e comportamentos (**métodos**).
- Ideia de representar coisas do mundo real como objetos no código.
- É composta por:
  - **Classe (molde para o objeto)**
    - **atributos = características do objeto;**
    - **métodos = ações do objeto;**
  - **Instância da classe (atribuir o molde a um objeto);**
  - **Encapsulamento (proteger os dados, tornar o mesmo privado);**
    - **atributos e métodos (não é 100% privado);**
  - **Polimorfismo trabalha com herança onde um método se comporta de um jeito com a super classe e comporta de outro com a subclasse;**
    - **herança pode herdar métodos sem sobrescrever;**





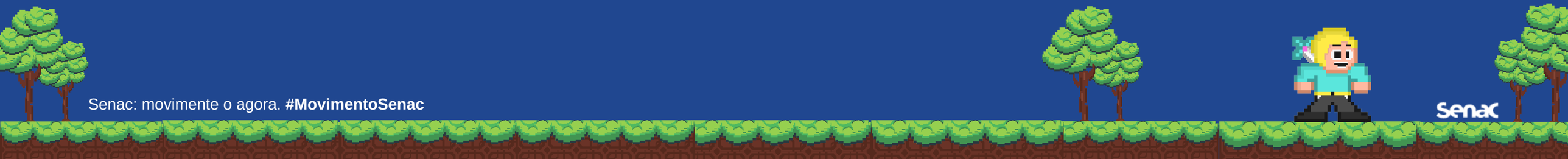
Orientação Objeto

Antes de implementarmos em código, vamos criar um diagrama;

- **Diagrama é utilizado para montagem do projeto e entender o funcionamento por meio de desenhos.**
- **De suma importância para documentos.**

Diagrama UML

- **Linguagem visual padrão;**
- **Classes;**
- **Casos de Uso:** interações entre usuários**;**
- **Atividade:** mostra o fluxo de ações(tipo fluxograma)**;**
- **Sequências:** Ordem dos eventos e trocas de mensagens;



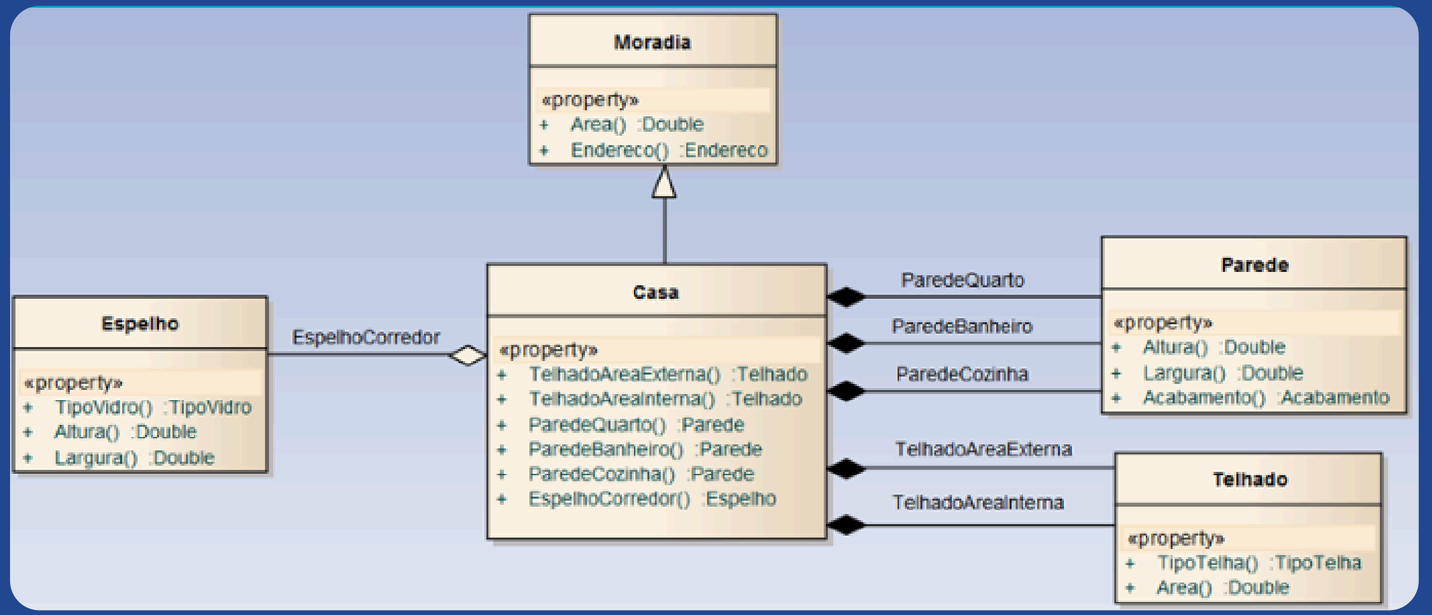
Orientação Objeto

Diagrama UML - Classes:

- **Representar sistemas orientados a objetos;**

Seus componentes:

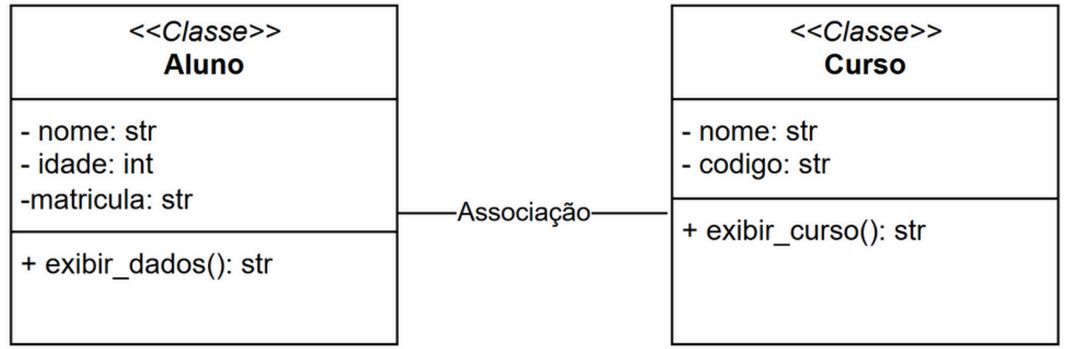
- **Classe;**
- **Atributos (variáveis da classe);**
  - **-** → privado;
  - **+** → público;
- **Métodos (ações da classe);**
- **Relacionamento (conexão das classes);**
  - **Herança/generalização:** classe herda de outra;
  - **Associação:** classe usa ou relaciona com outra;
  - **Agregação:** Classe composta por outra, vivem separadas;
  - **Composição:** Dependente principal, se uma morre a outra também;
  - **Depêndencia:** Depende temporariamente da outra;



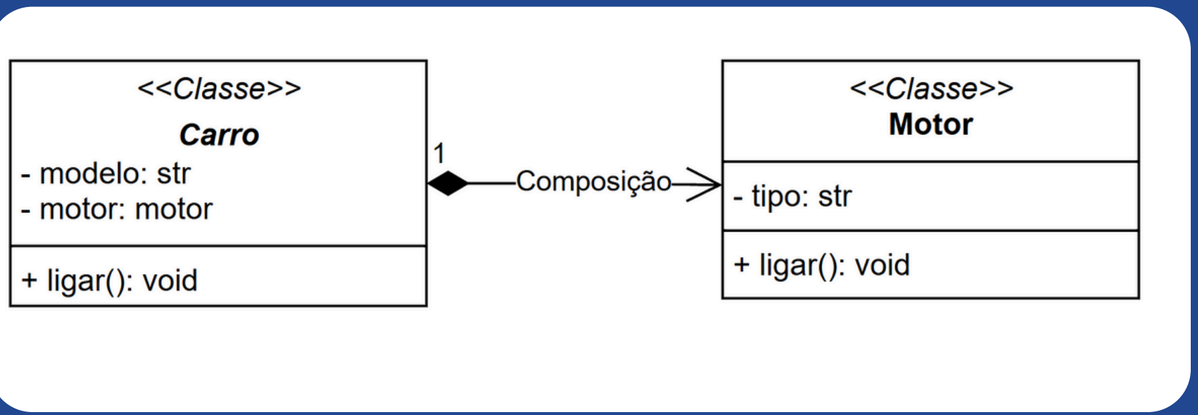
Orientação Objeto

Diagrama UML - Classes:

- **Exemplo 1 e 2:**
  - **Crie um diagrama de classe para um sistema de cadastro de alunos:**



  - **Crie um diagrama para de um carro:**



Orientação Objeto

Diagrama UML - Classes:

- **Exemplo 3, 4 e 5:**
  - **Crie um diagrama de classe de herança para Pessoa → Aluno, Professor;**
  - **Crie um diagrama de agregação de um Departamento e Funcionário;**
  - **Crie um diagrama de dependência de um relatório e impressora;**
- **https://app.diagrams.net/**



Orientação Objeto

Diagrama UML - Classes:

Atividade:

Crie quatro diagramas UML simples, um para cada tipo de relacionamento abaixo:

- **1. Herança**
  - Crie uma classe "Veículo" com atributos como marca e ano.
  - Crie duas subclasses: "Carro" e "Moto" com atributos específicos.
- **2. Composição**
  - Crie uma classe "Computador" que possui um objeto da classe "PlacaMãe".
  - A PlacaMãe não existe sem o Computador.
- **3. Agregação**
  - Crie uma classe "Time" e uma classe "Jogador".
  - Um Time possui Jogadores, mas os jogadores podem existir fora do time.
- **4. Dependência**
  - Crie uma classe "EditorTexto" que usa uma classe "CorretorOrtografico" apenas para corrigir um texto, sem armazená-lo.



Orientação Objeto

Diagrama UML - Classes:

Atividade:

Crie quatro diagramas UML simples, um para cada tipo de relacionamento abaixo:

- **6.Herança**
  - Crie uma classe "Funcionario" com os atributos nome e salario.
  - Crie duas subclasses: "Gerente" e "Operador", cada uma com um atributo próprio.
- **7. Composição**
  - Crie uma classe "Casa" que contém objetos da classe "Comodo".
  - Os cômodos não existem fora da casa.
- **8. Agregação**
  - Crie uma classe "Turma" e uma classe "Aluno".
  - Uma Turma possui Alunos, mas os Alunos podem existir fora da Turma.
- **9. Dependência**
  - Crie uma classe "AppPagamento" que usa a classe "GatewayPagamento" apenas para realizar o pagamento.



Orientação Objeto

POO - Programação orientada a Objeto:

- **A Programação Orientada a Objetos (POO)** é um paradigma de programação que organiza o código com base em objetos, que representam elementos do mundo real, como pessoas, produtos, contas bancárias, etc.
- Esses **objetos** são **criados a partir** de **classes**, que **funcionam como moldes com atributos** (características) **e métodos** (ações ou comportamentos).
- **Criada para combater a programação Procedural com códigos:**
  - 🔁 **Mais reutilizáveis**
  - 🧩 **Mais modulares**
  - 🔍 **Mais fáceis de entender e manter**
  - 🤝 **Mais próximos da lógica humana e do mundo real**

Orientação Objeto

POO - Programação orientada a Objeto:

- **Estrutura em Python:**

```python
class Veiculo:
    def __init__(self, marca, modelo, ano, cor):
        self.marca = marca
        self.modelo = modelo
        self.ano = ano
        self.cor = cor
        self.ligado = False

    def ligar(self):
        if not self.ligado:
            self.ligado = True
            print(f"{self.modelo} está agora ligado.")
        else:
            print(f"{self.modelo} já está ligado.")

    def desligar(self):
        if self.ligado:
            self.ligado = False
            print(f"{self.modelo} foi desligado.")
        else:
            print(f"{self.modelo} já está desligado.")

    def buzinar(self):
        print("Buzinando: Biiiiiip!")

    def __str__(self):
        status = "Ligado" if self.ligado else "Desligado"
        return f"{self.ano} {self.marca} {self.modelo} ({self.cor}) - {status}"
```

```python
from teste01 import Veiculo

moto = Veiculo("harley","fatbob", "2025", "verde")

moto.ligar()
moto.buzinar()
print(moto)
```

```
meteor 350 está agora ligado.
Buzinando: Biiiiiip!
2025 Royal Enfield meteor 350 (amarela) - Ligado
```

Orientação Objeto

POO - Programação orientada a Objeto:

- **Estrutura em Python com subclasse:**

```python
class Veiculo:
    def __init__(self, marca, modelo, ano, cor):
        self.marca = marca
        self.modelo = modelo
        self.ano = ano
        self.cor = cor
        self.ligado = False

    def ligar(self):
        if not self.ligado:
            self.ligado = True
            print(f"{self.modelo} está agora ligado.")
        else:
            print(f"{self.modelo} já está ligado.")

    def desligar(self):
        if self.ligado:
            self.ligado = False
            print(f"{self.modelo} foi desligado.")
        else:
            print(f"{self.modelo} já está desligado.")

    def buzinar(self):
        print("Buzinando: Biiiiiip!")

    def __str__(self):
        status = "Ligado" if self.ligado else "Desligado"
        return f"{self.ano} {self.marca} {self.modelo} ({self.cor}) - {status}"
```

```python
class Moto(Veiculo):
    def empinar(self):
        if self.ligado:
            print(f"{self.modelo} está empinando! ")
        else:
            print(f"{self.modelo} precisa estar ligada para empinar.")

class Carro(Veiculo):
    def abrir_porta_malas(self):
        print(f"Porta-malas do {self.modelo} foi aberto.")
```

```python
from teste01 import *

moto = Moto("harley","fatbob", "2025", "verde")

moto.ligado = True
moto.ligar()
moto.buzinar()
moto.empinar()
print(moto)
```

```
fatbob já está ligado.
Buzinando: Biiiiiip!
fatbob está empinando!
2025 harley fatbob (verde) - Ligado
```